E-Commerce Database System

Database Systems - Final Project Report

Riphah International University

Faculty of Computing

Course: Database Systems

Instructor: Ihtisham Ullah

Submitted by: Sibghatallah

Program: BSSE 4-2

Semester: Spring 2026

Submission Date: 10 May 2026

GitHub Repository: https://github.com/Sibghatallah-7/ecommerce-db-project

Table of Contents

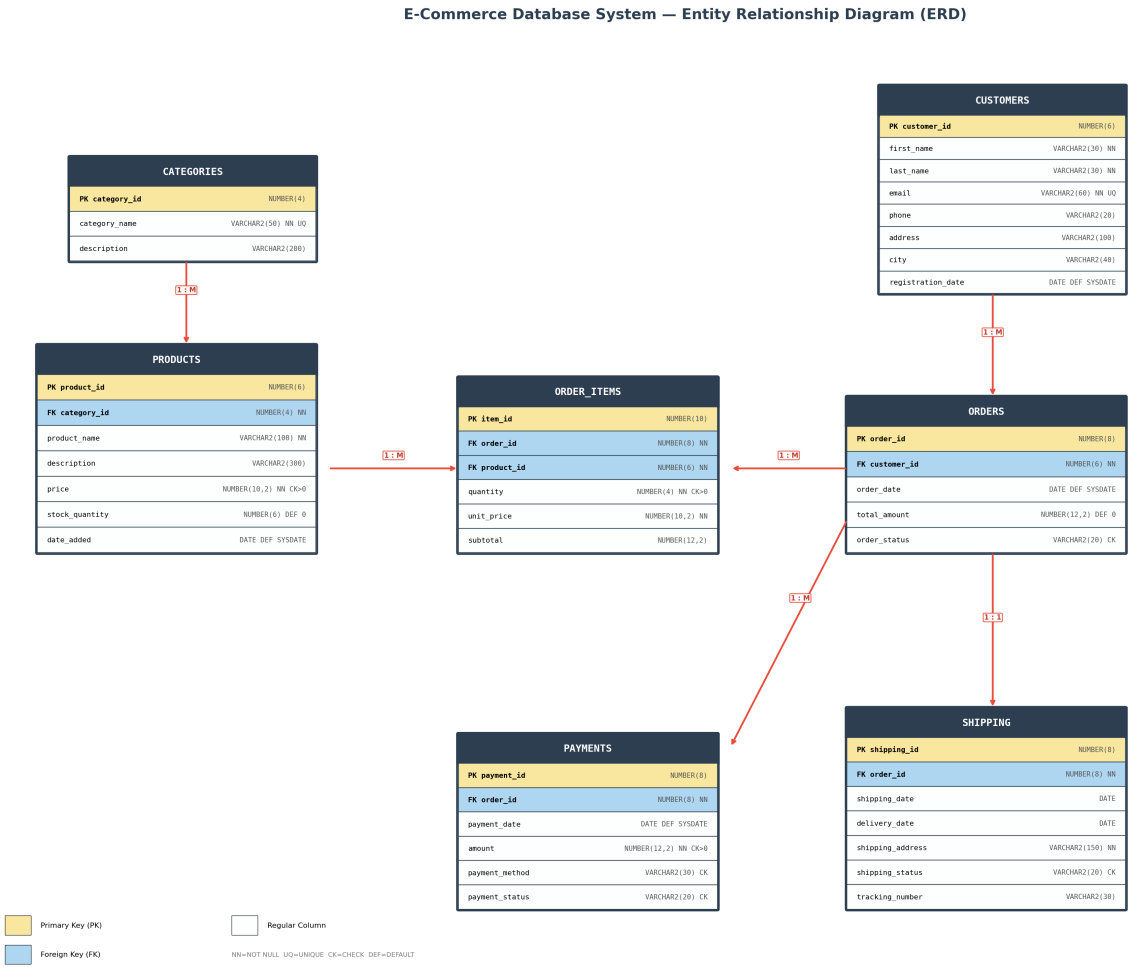
1. Introduction

This project implements a complete E-Commerce (Online Store) Database System using Oracle SQL. The system manages customers, products, product categories, orders, order items, payments, and shipping information. It demonstrates key database concepts including Entity Relationship Modeling, normalization to Third Normal Form (3NF), SQL DDL (Data Definition Language), DML (Data Manipulation Language), and advanced querying techniques such as joins, subqueries, and aggregation functions.

The E-Commerce system supports the following operations:
- Customers can register and browse a catalog of products organized by categories.
- Customers place orders containing one or more products (via the order_items bridge table).
- Each order has an associated payment record tracking the payment method and status.
- Each order has a shipping record tracking dispatch, delivery, and tracking info.
- The database provides views for customer summaries, order breakdowns, category revenue, product catalogs, and shipment tracking.

2. Entity Relationship Diagram (ERD)

The ERD below shows all 7 entities, their attributes, primary keys (PK), foreign keys (FK), constraints, and the relationships between tables with their cardinality (1:M or 1:1).



Relationships Summary

```
CATEGORIES -> PRODUCTS          [1:M]  One category contains many products
CUSTOMERS -> ORDERS             [1:M]  One customer can place many orders
ORDERS -> ORDER_ITEMS           [1:M]  One order contains many order items
PRODUCTS -> ORDER_ITEMS         [1:M]  One product appears in many order items
ORDERS -> PAYMENTS              [1:M]  One order can have multiple payments
ORDERS -> SHIPPING              [1:1]  One order has one shipping record
```

3. Schema Objects

3.1 Tables, Columns, Keys & Constraints

The database consists of 7 tables. Below is the CREATE TABLE DDL for each:

Table: CATEGORIES

Column	Data Type	Constraints
category_id	NUMBER(4)	Primary Key
category_name	VARCHAR2(50)	NOT NULL, UNIQUE
description	VARCHAR2(200)	-

```
CREATE TABLE categories (
    category_id     NUMBER(4)       PRIMARY KEY,
    category_name   VARCHAR2(50)    NOT NULL UNIQUE,
    description     VARCHAR2(200)
);
```

Table: PRODUCTS

Column	Data Type	Constraints
product_id	NUMBER(6)	Primary Key
category_id	NUMBER(4)	Foreign Key -> CATEGORIES
product_name	VARCHAR2(100)	NOT NULL
description	VARCHAR2(300)	-
price	NUMBER(10,2)	NOT NULL, CHECK > 0
stock_quantity	NUMBER(6)	DEFAULT 0, CHECK >= 0
date_added	DATE	DEFAULT SYSDATE

```
CREATE TABLE products (
    product_id      NUMBER(6)       PRIMARY KEY,
    category_id     NUMBER(4)       NOT NULL,
    product_name    VARCHAR2(100)   NOT NULL,
    description     VARCHAR2(300),
    price           NUMBER(10,2)    NOT NULL CHECK (price > 0),
    stock_quantity  NUMBER(6)       DEFAULT 0 CHECK (stock_quantity >= 0),
    date_added      DATE            DEFAULT SYSDATE,
    FOREIGN KEY (category_id) REFERENCES categories(category_id)
);
```

Table: CUSTOMERS

Column	Data Type	Constraints
customer_id	NUMBER(6)	Primary Key
first_name	VARCHAR2(30)	NOT NULL
last_name	VARCHAR2(30)	NOT NULL
email	VARCHAR2(60)	NOT NULL, UNIQUE
phone	VARCHAR2(20)	-
address	VARCHAR2(100)	-
city	VARCHAR2(40)	-
registration_date	DATE	DEFAULT SYSDATE

```
CREATE TABLE customers (
    customer_id       NUMBER(6)       PRIMARY KEY,
    first_name        VARCHAR2(30)    NOT NULL,
    last_name         VARCHAR2(30)    NOT NULL,
    email             VARCHAR2(60)    NOT NULL UNIQUE,
    phone             VARCHAR2(20),
    address           VARCHAR2(100),
    city              VARCHAR2(40),
    registration_date DATE            DEFAULT SYSDATE
);
```

Table: ORDERS

Column	Data Type	Constraints
order_id	NUMBER(8)	Primary Key
customer_id	NUMBER(6)	Foreign Key -> CUSTOMERS
order_date	DATE	DEFAULT SYSDATE
total_amount	NUMBER(12,2)	DEFAULT 0
order_status	VARCHAR2(20)	CHECK IN list, DEFAULT Pending

```
CREATE TABLE orders (
    order_id        NUMBER(8)       PRIMARY KEY,
    customer_id     NUMBER(6)       NOT NULL,
    order_date      DATE            DEFAULT SYSDATE,
    total_amount    NUMBER(12,2)    DEFAULT 0,
    order_status    VARCHAR2(20)    DEFAULT 'Pending'
                    CHECK (order_status IN ('Pending','Processing',
                           'Shipped','Delivered','Cancelled')),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

Table: ORDER_ITEMS

Column	Data Type	Constraints
item_id	NUMBER(10)	Primary Key
order_id	NUMBER(8)	Foreign Key -> ORDERS
product_id	NUMBER(6)	Foreign Key -> PRODUCTS
quantity	NUMBER(4)	NOT NULL, CHECK > 0
unit_price	NUMBER(10,2)	NOT NULL
subtotal	NUMBER(12,2)	-

```
CREATE TABLE order_items (
    item_id         NUMBER(10)      PRIMARY KEY,
    order_id        NUMBER(8)       NOT NULL,
    product_id      NUMBER(6)       NOT NULL,
    quantity        NUMBER(4)       NOT NULL CHECK (quantity > 0),
    unit_price      NUMBER(10,2)    NOT NULL,
    subtotal        NUMBER(12,2),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

Table: PAYMENTS

Column	Data Type	Constraints
payment_id	NUMBER(8)	Primary Key
order_id	NUMBER(8)	Foreign Key -> ORDERS
payment_date	DATE	DEFAULT SYSDATE
amount	NUMBER(12,2)	NOT NULL, CHECK > 0
payment_method	VARCHAR2(30)	CHECK IN list
payment_status	VARCHAR2(20)	CHECK IN list, DEFAULT Completed

```
CREATE TABLE payments (
    payment_id      NUMBER(8)       PRIMARY KEY,
    order_id        NUMBER(8)       NOT NULL,
    payment_date    DATE            DEFAULT SYSDATE,
    amount          NUMBER(12,2)    NOT NULL CHECK (amount > 0),
    payment_method  VARCHAR2(30)    CHECK (payment_method IN
                    ('Credit Card','Debit Card','Cash on Delivery',
                     'Bank Transfer','PayPal')),
    payment_status  VARCHAR2(20)    DEFAULT 'Completed'
                    CHECK (payment_status IN ('Completed','Pending',
                           'Failed','Refunded')),
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
);
```

Table: SHIPPING

Column	Data Type	Constraints
shipping_id	NUMBER(8)	Primary Key
order_id	NUMBER(8)	Foreign Key -> ORDERS
shipping_date	DATE	-
delivery_date	DATE	-
shipping_address	VARCHAR2(150)	NOT NULL
shipping_status	VARCHAR2(20)	CHECK IN list, DEFAULT Pending
tracking_number	VARCHAR2(30)	-

```
 CREATE TABLE shipping (
     shipping_id       NUMBER(8)       PRIMARY KEY,
     order_id          NUMBER(8)       NOT NULL,
     shipping_date     DATE,
     delivery_date     DATE,
     shipping_address  VARCHAR2(150)   NOT NULL,
     shipping_status   VARCHAR2(20)    DEFAULT 'Pending'
                       CHECK (shipping_status IN ('Pending','Dispatched',
                              'In Transit','Delivered','Returned')),
     tracking_number   VARCHAR2(30),
     FOREIGN KEY (order_id) REFERENCES orders(order_id)
 );
```

3.2 Normalization (up to 3NF)

The database schema is normalized to Third Normal Form (3NF). Below is the normalization analysis:

First Normal Form (1NF):

- All columns contain atomic (indivisible) values. For example, customer name is split into first_name and last_name rather than a single name field.
- Each table has a primary key that uniquely identifies every row.
- There are no repeating groups or arrays in any column.

Second Normal Form (2NF):

- The schema satisfies 1NF.
- All non-key attributes are fully functionally dependent on the entire primary key.
- Since all tables use single-column primary keys, there are no partial dependencies.
- The ORDER_ITEMS table uses item_id as PK (not a composite of order_id + product_id), and order_id and product_id are foreign keys, so all attributes depend on item_id alone.

Third Normal Form (3NF):

- The schema satisfies 2NF.
- There are no transitive dependencies. Every non-key attribute depends directly on the primary key and nothing else.
- Category information is stored in a separate CATEGORIES table, not repeated in PRODUCTS.
- Customer information is stored in CUSTOMERS, not repeated in ORDERS.
- Payment and Shipping details are in their own tables, linked by order_id FK.
- Product price is stored in PRODUCTS, and unit_price in ORDER_ITEMS captures the price at the time of purchase (historical price), which is a separate fact.

3.3 Relationships

The following relationships exist between the tables:

CATEGORIES (1) --> PRODUCTS (M)

One category can contain many products. Each product belongs to exactly one category. The category_id foreign key in PRODUCTS references CATEGORIES.

CUSTOMERS (1) --> ORDERS (M)

One customer can place many orders over time. Each order belongs to exactly one customer. The customer_id foreign key in ORDERS references CUSTOMERS.

ORDERS (1) --> ORDER_ITEMS (M)

One order can contain many line items (products). Each order item belongs to exactly one order. The order_id foreign key in ORDER_ITEMS references ORDERS.

PRODUCTS (1) --> ORDER_ITEMS (M)

One product can appear in many order items across different orders. The product_id foreign key in ORDER_ITEMS references PRODUCTS. This creates a Many-to-Many relationship between ORDERS and PRODUCTS via ORDER_ITEMS.

ORDERS (1) --> PAYMENTS (M)

One order can have one or more payment records (e.g., partial payments, refunds). The order_id foreign key in PAYMENTS references ORDERS.

ORDERS (1) --> SHIPPING (1)

Each order has one shipping record tracking its dispatch and delivery status. The order_id foreign key in SHIPPING references ORDERS.

3.4 Views

Five views are created to simplify common data retrieval operations:

View: vw_customer_order_summary

Displays each customer with their total number of orders and total amount spent.

```
CREATE OR REPLACE VIEW vw_customer_order_summary AS
SELECT c.customer_id,
       c.first_name || ' ' || c.last_name AS customer_name,
       c.email, c.city,
       COUNT(o.order_id) AS total_orders,
       NVL(SUM(o.total_amount), 0) AS total_spent
FROM customers c
LEFT OUTER JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name, c.email, c.city;
```

View: vw_product_catalog

Displays product details with their category name for browsing.

```
CREATE OR REPLACE VIEW vw_product_catalog AS
SELECT p.product_id, p.product_name, c.category_name,
       p.price, p.stock_quantity, p.date_added
FROM products p
JOIN categories c ON p.category_id = c.category_id;
```

View: vw_order_details

Displays full order breakdown with customer, product, and category info.

```
CREATE OR REPLACE VIEW vw_order_details AS
SELECT o.order_id, o.order_date,
       c.first_name || ' ' || c.last_name AS customer_name,
       p.product_name, cat.category_name,
       oi.quantity, oi.unit_price, oi.subtotal, o.order_status
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
JOIN categories cat ON p.category_id = cat.category_id;
```

View: vw_category_revenue

Displays total revenue generated by each product category (excluding cancelled).

```
CREATE OR REPLACE VIEW vw_category_revenue AS
SELECT cat.category_id, cat.category_name,
       COUNT(DISTINCT oi.item_id) AS items_sold,
       NVL(SUM(oi.subtotal), 0) AS total_revenue
FROM categories cat
LEFT OUTER JOIN products p ON cat.category_id = p.category_id
LEFT OUTER JOIN order_items oi ON p.product_id = oi.product_id
LEFT OUTER JOIN orders o ON oi.order_id = o.order_id
  AND o.order_status <> 'Cancelled'
GROUP BY cat.category_id, cat.category_name;
```

View: vw_shipping_tracker

Displays shipping status with customer and order details.

```
CREATE OR REPLACE VIEW vw_shipping_tracker AS
SELECT s.shipping_id, s.tracking_number, o.order_id,
       c.first_name || ' ' || c.last_name AS customer_name,
       s.shipping_address, s.shipping_date, s.delivery_date,
       s.shipping_status, o.total_amount
FROM shipping s
JOIN orders o ON s.order_id = o.order_id
JOIN customers c ON o.customer_id = c.customer_id;
```

4. Queries

4.1 Joins

The following queries demonstrate different types of SQL joins:

Q1. INNER JOIN - Orders with Customer Name

```
SELECT o.order_id,
       c.first_name || ' ' || c.last_name AS customer_name,
       o.order_date, o.total_amount, o.order_status
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;
```

Q2. 3-Way INNER JOIN - Order Items with Product & Category

```
SELECT oi.item_id, o.order_id, p.product_name,
       cat.category_name, oi.quantity, oi.unit_price, oi.subtotal
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
JOIN products p ON oi.product_id = p.product_id
JOIN categories cat ON p.category_id = cat.category_id;
```

Q3. LEFT OUTER JOIN - All Customers Including Those With No Orders

```
SELECT c.customer_id,
       c.first_name || ' ' || c.last_name AS customer_name,
       c.city, o.order_id, o.total_amount
FROM customers c
LEFT OUTER JOIN orders o ON c.customer_id = o.customer_id
ORDER BY c.customer_id;
```

Q4. RIGHT OUTER JOIN - All Products Including Those Never Ordered

```
SELECT p.product_id, p.product_name, p.price,
       oi.order_id, oi.quantity
FROM order_items oi
RIGHT OUTER JOIN products p ON oi.product_id = p.product_id
ORDER BY p.product_id;
```

Q5. FULL OUTER JOIN - All Orders and All Shipping Records

```
SELECT o.order_id, o.order_status,
       s.shipping_id, s.shipping_status, s.tracking_number
FROM orders o
FULL OUTER JOIN shipping s ON o.order_id = s.order_id;
```

Q6. SELF JOIN - Customers From the Same City

```
SELECT a.first_name || ' ' || a.last_name AS customer_1,
       b.first_name || ' ' || b.last_name AS customer_2,
       a.city
FROM customers a, customers b
WHERE a.city = b.city
  AND a.customer_id < b.customer_id;
```

4.2 Subqueries

The following queries demonstrate single-row, multi-row, and correlated subqueries:

Q7. Single-Row Subquery - Products Above Average Price

```
SELECT product_id, product_name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

Q8. Single-Row Subquery - Customer With Highest Order

```
SELECT c.first_name || ' ' || c.last_name AS customer_name,
       o.order_id, o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.total_amount = (SELECT MAX(total_amount) FROM orders);
```

Q9. Multi-Row Subquery (IN) - Customers Who Ordered Electronics

```
SELECT DISTINCT c.customer_id,
       c.first_name || ' ' || c.last_name AS customer_name
FROM customers c
WHERE c.customer_id IN (
    SELECT o.customer_id
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    JOIN products p ON oi.product_id = p.product_id
    WHERE p.category_id = (SELECT category_id FROM categories
                           WHERE category_name = 'Electronics')
);
```

Q10. Multi-Row Subquery (ANY) - Products Cheaper Than ANY Electronics

```
SELECT product_id, product_name, price
FROM products
WHERE price < ANY (
    SELECT price FROM products WHERE category_id = 1
);
```

Q11. Multi-Row Subquery (ALL) - Products Cheaper Than ALL Electronics

```
SELECT product_id, product_name, price
FROM products
WHERE price < ALL (
    SELECT price FROM products WHERE category_id = 1
);
```

Q12. Subquery in HAVING - Categories With Avg Price Above Overall Avg

```
SELECT cat.category_name, AVG(p.price) AS avg_price
FROM products p
JOIN categories cat ON p.category_id = cat.category_id
GROUP BY cat.category_name
HAVING AVG(p.price) > (SELECT AVG(price) FROM products);
```

Q13. Correlated Subquery - Each Customer's Most Recent Order

```
SELECT c.first_name || ' ' || c.last_name AS customer_name,
       o.order_id, o.order_date, o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.order_date = (
    SELECT MAX(o2.order_date)
    FROM orders o2
    WHERE o2.customer_id = o.customer_id
);
```

4.3 Aggregation Functions

The following queries demonstrate COUNT, SUM, AVG, MIN, MAX, GROUP BY, and HAVING:

Q14. COUNT - Total Orders Per Status

```
SELECT order_status, COUNT(*) AS order_count
FROM orders
GROUP BY order_status
ORDER BY order_count DESC;
```

Q15. SUM - Total Revenue (Excluding Cancelled)

```
SELECT SUM(total_amount) AS total_revenue
FROM orders
WHERE order_status <> 'Cancelled';
```

Q16. AVG - Average Order Value Per Customer

```
SELECT c.first_name || ' ' || c.last_name AS customer_name,
       COUNT(o.order_id) AS total_orders,
       ROUND(AVG(o.total_amount), 2) AS avg_order_value
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.first_name, c.last_name
ORDER BY avg_order_value DESC;
```

Q17. MIN/MAX - Cheapest & Most Expensive Per Category

```
SELECT cat.category_name,
       MIN(p.price) AS cheapest_product,
       MAX(p.price) AS most_expensive
FROM products p
JOIN categories cat ON p.category_id = cat.category_id
GROUP BY cat.category_name;
```

Q18. GROUP BY with HAVING - Categories With Revenue > 50,000

```
SELECT cat.category_name,
       SUM(oi.subtotal) AS total_revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
JOIN categories cat ON p.category_id = cat.category_id
JOIN orders o ON oi.order_id = o.order_id
WHERE o.order_status <> 'Cancelled'
GROUP BY cat.category_name
HAVING SUM(oi.subtotal) > 50000
ORDER BY total_revenue DESC;
```

Q19. Nested Aggregation - Category With Highest Avg Price

```
SELECT category_name, avg_price
FROM (
    SELECT cat.category_name,
           ROUND(AVG(p.price), 2) AS avg_price
    FROM products p
    JOIN categories cat ON p.category_id = cat.category_id
    GROUP BY cat.category_name
    ORDER BY avg_price DESC
)
WHERE ROWNUM = 1;
```

Q20. COUNT DISTINCT - Unique Customers Who Ordered

```
SELECT COUNT(DISTINCT customer_id) AS unique_customers
FROM orders;
```

Q21. Revenue By Payment Method

```
SELECT payment_method,
       COUNT(*) AS transaction_count,
       SUM(amount) AS total_collected
FROM payments
WHERE payment_status = 'Completed'
GROUP BY payment_method
ORDER BY total_collected DESC;
```

Q22. Monthly Sales Report

```
SELECT TO_CHAR(order_date, 'YYYY-MM') AS month,
       COUNT(*) AS total_orders,
       SUM(total_amount) AS monthly_revenue
FROM orders
WHERE order_status <> 'Cancelled'
GROUP BY TO_CHAR(order_date, 'YYYY-MM')
ORDER BY month;
```

Q23. Top 3 Best-Selling Products

```
SELECT product_name, total_qty_sold
FROM (
    SELECT p.product_name,
           SUM(oi.quantity) AS total_qty_sold
    FROM order_items oi
    JOIN products p ON oi.product_id = p.product_id
    JOIN orders o ON oi.order_id = o.order_id
    WHERE o.order_status <> 'Cancelled'
    GROUP BY p.product_name
    ORDER BY total_qty_sold DESC
)
WHERE ROWNUM <= 3;
```

Q24. Customers Who Never Placed an Order

```
SELECT c.customer_id,
       c.first_name || ' ' || c.last_name AS customer_name,
       c.email
FROM customers c
WHERE c.customer_id NOT IN (
    SELECT DISTINCT customer_id FROM orders
);
```

5. Sample Data

The database is populated with realistic sample data to demonstrate all features:

- 5 Categories (Electronics, Clothing, Books, Home & Kitchen, Sports & Outdoors)
- 10 Products spread across all categories
- 8 Customers with Pakistani names and cities
- 10 Orders with various statuses (Pending, Processing, Shipped, Delivered, Cancelled)
- 15 Order Items linking orders to products
- 10 Payments with different methods (Credit Card, Debit Card, COD, Bank Transfer, PayPal)
- 9 Shipping records with tracking numbers and statuses

All INSERT statements are provided in the file sql/02_insert_data.sql in the GitHub repository.

6. GitHub Repository

The complete source code for this project is available on GitHub:

https://github.com/Sibghatallah-7/ecommerce-db-project

Repository structure:

```
ecommerce-db-project/
|-- README.md
|-- sql/
|   |-- 01_create_tables.sql    (DDL - Table creation)
|   |-- 02_insert_data.sql      (DML - Sample data)
|   |-- 03_views.sql            (Views)
|   |-- 04_queries.sql          (Joins, Subqueries, Aggregation)
|   |-- 05_drop_tables.sql      (Cleanup script)
|-- diagrams/
|   |-- ERD_ECommerce.png       (ERD diagram)
|-- report/
    |-- ECommerce_DB_Report.pdf (This report)
```